AI Intellicore CMP

Intellicore CMP — Setup & Developer Guide

A Searce Product · Powered by CSRE · Managed cloud intelligence

Intellicore CMP — Setup & Developer Guide

Prerequisites

- Node.js 20+
- Python 3.11+
- Docker & Docker Compose
- Git

Local Development

1. Clone and install

```
git clone https://github.com/nikhiljohn/grafana_pricing_repo.git
cd grafana_pricing_repo
```

2. Frontend

```
cd frontend
npm install
npm run dev
# → http://localhost:3000
```

The frontend runs with mock data by default — no backend needed for UI development.

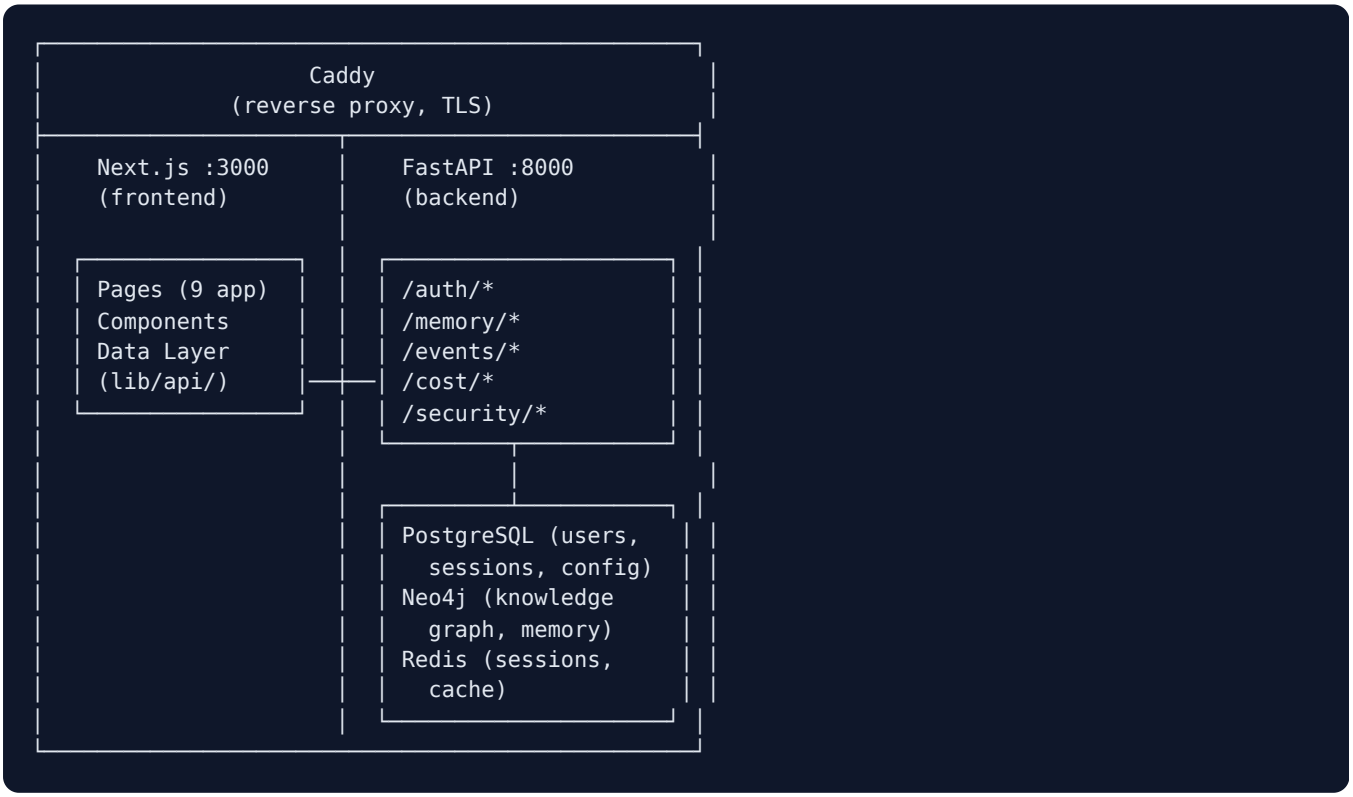
3. Backend

```
cd backend
python -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"

# Start dependencies
docker compose up -d postgres neo4j redis

# Run migrations
psql $DATABASE_URL < ../postgres/init/01-schema.sql

# Create admin user
python scripts/create_admin.py --email admin@searce.com --password <password>

# Seed demo data
python scripts/seed_demo.py

# Start server
uvicorn app.main:app --reload --port 8000
```

4. Full stack (Docker Compose)

```
docker compose up --build
# Frontend: http://localhost:3000
# Backend:  http://localhost:8000
# Neo4j:    http://localhost:7474
```

Architecture



Frontend Data Layer

Current State

All 9 app pages render hardcoded data from `const` arrays. A data abstraction layer has been created but is **not yet wired** to the pages.

Data Layer Files (`frontend/src/lib/api/`)

File	Purpose
`types.ts`	25 TypeScript interfaces for all data models
`mock.ts`	26 mock endpoints mapping paths to realistic data
`client.ts`	`apiFetch<T>(endpoint)` — mock now, real API when `NEXT_PUBLIC_API_URL` is set
`hooks.ts`	`useApiData<T>(endpoint, initialData)` — React hook with loading/error state
`index.ts`	Barrel export for clean imports

How to Wire a Page

Before (hardcoded):

```
const scores = [
  { pillar: "CloudOps", score: 94, status: "healthy", note: "" },
  // ...
];

export default function Page() {
  return <div>{scores.map(s => <Card key={s.pillar} {...s} />)}</div>;
}
```

After (data layer):

```
import { useApiData, OpsScore } from "@/lib/api";

export default function Page() {
  const { data: scores, loading } = useApiData<OpsScore[]>(
    "/command-center/scores",
    []
  );

  if (loading) return <Skeleton />;
  return <div>{scores.map(s => <Card key={s.pillar} {...s} />)}</div>;
}
```

Switching to Real API

Set the environment variable:

```
NEXT_PUBLIC_API_URL=https://your-domain.com/api
```

Then uncomment the `fetch()` block in `client.ts` and remove the mock import.

Legacy API Client (`lib/api.ts`)

A separate file handles auth and Memory backend calls. Used by: - `TopBar` — fetches `/auth/me` for user display - `LoginForm` — posts to `/auth/login` and `/auth/totp/verify`

This file talks to the real backend and is functional.

Backend API Reference

Auth (`/auth`)

Endpoint	Method	Body	Response
`/auth/login`	POST	`{email, password}`	`{stage: "totp_setup"|"totp_required", qr_code?, secret?}`
`/auth/totp/verify`	POST	`{code, remember}`	`{ok: true}`
`/auth/session/check`	GET	—	`{valid: true, email}`
`/auth/logout`	POST	—	`{ok: true}`
`/auth/me`	GET	—	`{email, tenant_id}`

Memory (`/memory`)

Endpoint	Method	Body	Response
`/memory/timeline`	GET	Query: `limit`, `category`, `severity`	`{events[], total, cursor}`
`/memory/patterns`	GET	Query: `min_confidence`	`{patterns[], total}`
`/memory/event/{id}`	GET	—	`{event, causes[], caused[]}`
`/memory/chat`	POST	`{question}`	`{answer, cited_event_ids[], cited_pattern_ids[], cypher_used}`

Events (`/events`)

Endpoint	Method	Body	Response
`/events/ingest`	POST	`{events[]}`	`{ingested, graph_nodes_created}`

Cost (`/cost`) — Stub

Endpoint	Method	Response
`/cost/summary`	GET	Placeholder data

Security (`/security`) — Stub

Endpoint	Method	Response
`/security/summary`	GET	Placeholder data

Health (`/`)

Endpoint	Method	Response
`/health`	GET	`{status: "ok"}`
`/ready`	GET	`{postgres, neo4j, redis}` connection status

CI/CD Pipelines

GitHub Actions Workflows

Workflow	Trigger	What It Does
`ci.yml`	Push/PR to `main` / `staging`	Lint + build frontend, syntax-check backend
`deploy-staging.yml`	Push to `staging`	Package → SCP to staging VM → docker compose up
`deploy-production.yml`	Push to `main`	Build test → deploy to prod VM → health check → auto-rollback on failure
`deploy-customer.yml`	Manual dispatch	Provision new GCP VM → install Docker → deploy → generate credentials

Required Secrets

Secret/Variable	Where	Purpose
`GCP_SA_KEY`	Secret	GCP service account JSON (Compute Admin, SSH)
`GCP_PROJECT_ID`	Variable	GCP project ID
`VM_NAME`	Variable	Target VM name (optional, has defaults)
`VM_ZONE`	Variable	GCP zone (optional, defaults to asia-south1-a)

Per-Customer Deployment

Run via GitHub Actions → "Deploy Customer Environment" → fill inputs: - **Customer name**: lowercase, no spaces (e.g., `acme-corp`) - **GCP Project ID**: customer's GCP project - **Region**: deployment region - **Machine type**: VM size - **Admin email**: first admin user's email

The workflow provisions a VM, configures Docker, deploys, and outputs: - URL (via sslip.io for auto-TLS) - Admin credentials

---

Database Schema

PostgreSQL

```
users       — id, email, hashed_password, totp_secret, tenant_id
tenants     — id, name, slug, created_at
sessions    — id, user_id, token_hash, expires_at
events      — id, tenant_id, timestamp, type, category, severity, title, summary
patterns    — id, tenant_id, title, description, category, confidence
```

Neo4j

```
(:Event {id, type, category, severity, title, summary, timestamp})
(:Pattern {id, title, description, confidence})
(:Resource {id, type, provider, region})

(:Event)-[:CAUSED]->(:Event)
(:Event)-[:AFFECTED]->(:Resource)
(:Pattern)-[:MATCHES]->(:Event)
```

Roadmap to Production

Phase 1: Wire Data Layer (Current Priority)

- Replace hardcoded `const` arrays in all 9 pages with `useApiData()` hooks
- Add loading skeletons for data fetch states
- Add error boundaries for API failures

Phase 2: Build Backend Endpoints

- Implement `/cloudops/*` endpoints (workloads, incidents, metrics)
- Implement `/finops/*` endpoints (costs, anomalies, optimizations)
- Implement `/secops/*` endpoints (findings, IAM, compliance)
- Implement `/devops/*` endpoints (changes, orchestration, patches)
- Implement `/aiops/*` endpoints (agents, analysis, governance)
- Expand `/memory/*` with pillar-specific queries

Phase 3: Connect to Real Cloud APIs

- GCP Cloud Monitoring / Operations Suite integration
- AWS CloudWatch integration
- Cost data from GCP Billing / AWS Cost Explorer
- Security findings from Wiz / SCC Premium
- IAM data from GCP IAM / AWS IAM

Phase 4: Action Buttons

- Wire "Apply Fix" to backend remediation endpoints
- Wire "Auto-fix" to automated remediation scripts
- Wire "Acknowledge" to alert management
- Wire AIOps query to `/memory/chat`
- Implement orchestration approval workflow

Phase 5: Real-time Features

- WebSocket for live alert updates
- Server-Sent Events for AI agent status
- Background job processing for cloud API polling
